Renewable Energy Plant System (REPS)

Final Report - Part A

Scala functional programming project report and design diagrams

Item	Details
Project name	Renewable Energy Plant System (REPS)
Language / build	Scala 2.13.12, sbt project
Repository	https://github.com/arsoykot/reps
Main entry point	reps.ui.REPSApp
Data file	data/energy_readings.csv

1. Introduction

The Renewable Energy Plant System (REPS) is a command-line Scala application for monitoring a plant with solar, wind, and hydro energy sources. The system records production readings, stores them in a CSV file, filters and searches historical data, calculates statistical summaries, and generates alerts for abnormal operating conditions.

The implementation follows a functional style where the core domain data is immutable, transformations return new values instead of changing old ones, and validation/error handling is represented with Option and Either.

2. GitHub repository

The codebase should be submitted through a GitHub repository. Before uploading this report to Lithium, replace the placeholder below with the actual repository link.

Required item	Submission value
GitHub repository URL	https://github.com/arsoykot/reps
Branch to assess	main
Run command	sbt run
Expected data location	data/energy_readings.csv

3. System purpose and main features

- Record energy readings for Solar, Wind, and Hydro sources with operational status.
- Load and save readings using CSV persistence.
- View a plant overview showing total production, storage level, and latest reading per source.
- Filter data by hour, day, week, or month.
- Analyse data using mean, median, mode, range, midrange, minimum, and maximum.
- Search readings by keyword or kWh output range.
- Sort readings by timestamp, output, or source.
- Generate warnings and critical alerts for low output, offline sources, degradation, and repeated zero output.
- Optionally fetch recent public production data from the Fingrid API using an API key.

4. Architecture overview

The system is divided into model, service, analysis, I/O, and UI responsibilities. REPSApp controls the menu loop and delegates work to services. Pure services such as DataService, AnalysisService, AlertService, Validation, and Display operate mainly by receiving values and returning new values or formatted strings. CsvIO and FingridAPI are intentionally separated because they perform external I/O.

Package / file	Responsibility
reps.models.Models	Domain model: EnergyReading, EnergySource, SourceStatus, TimeFilter, SortConfig, AnalysisSummary
reps.models.Alerts	Alert model, severity types, and default threshold configuration
reps.services.PlantMonitor	Immutable plant state creation, loading, recording readings, storage updates, production summary
reps.services.DataService	Filtering, sorting, and searching readings
reps.analysis.AnalysisService	Statistical calculations and grouped summaries
reps.services.AlertService	Alert detection rules

reps.services.Validation	Composable input validation with Either
reps.io.CsvIO	CSV loading, appending, and writing
reps.io.FingridAPI	Fetch and parse Fingrid Open Data API values
reps.ui.Display	Pure formatting of tables, summaries, alerts, and overview text
reps.ui.REPSApp	CLI entry point, recursive main loop, and menu handlers

5. Functional programming techniques used

Technique	How it is used in the code
Immutability	PlantState and all main domain values are case classes. Updating readings uses state.copy rather than mutating the existing state.
Sealed traits and pattern matching	EnergySource, SourceStatus, TimeFilter, SortField, and AlertSeverity are sealed hierarchies matched exhaustively.
Higher-order functions	DataService.filterBy accepts predicates; AnalysisService.applyAll applies labelled statistical functions.
Currying	DataService filters return List[EnergyReading] => List[EnergyReading]; AnalysisService.aboveThreshold(threshold)(reading) is curried.
Option	Statistical functions return None for empty input, avoiding invalid calculations.
Either	Validation and I/O-heavy service methods return Left(error) or Right(value), which supports for-comprehension composition.
Recursion	REPSApp.mainLoop is tail-recursive and threads the current PlantState through each menu action.
Pure formatting	Display functions return strings instead of printing directly, keeping output formatting separate from I/O.

6. Data model

The central record is EnergyReading. Each reading has an identifier, source, timestamp, output value, and status.

Entity	Important fields / variants	Purpose
EnergyReading	id, source, timestamp, outputKWh, status	One production measurement from a plant source.
EnergySource	Solar, Wind, Hydro	Supported renewable source categories.
SourceStatus	Operational, Degraded, Offline, Maintenance	Operational state of a source at the time of reading.
PlantState	readings, storageKWh, maxStorageKWh, dataFilePath	Immutable snapshot of the plant state.
AnalysisSummary	count, mean, median, mode, range, midrange, min, max	Calculated statistics for all readings or one source.
Alert	id, source, severity, message, timestamp, resolved	Operator-facing warning or critical condition.

AlertThreshold	minOutputKWh, criticalMinOutputKWh, degradedRatioThreshold	Per-source alert rule configuration.

7. Main use cases

7.1 Record an energy reading

The user enters a source, output value, and status. REPSApp validates the input, converts source/status strings to domain values, and calls PlantMonitor.recordReading. PlantMonitor creates a new EnergyReading, appends it to the CSV file through CsvIO, and returns a new PlantState containing the added reading.

7.2 View, filter, search, and sort readings

Readings are held in the current PlantState. DataService provides pure functions for filtering by source and time, searching by keyword or output range, and sorting by timestamp, output, or source.

7.3 Analyse readings

AnalysisService receives a list of readings, extracts outputKWh values, and calculates mean, median, mode, range, midrange, minimum, and maximum. Empty datasets return None, and successful calculations are wrapped in AnalysisSummary.

7.4 Generate alerts

AlertService groups readings by source and checks each group against default thresholds. It generates critical alerts for offline sources or repeated zero output, and warnings for low output or high degradation ratio.

7.5 Fetch Fingrid data

If the user provides a Fingrid API key, FingridAPI fetches recent wind, hydro, and solar values, maps them to EnergyReading objects, merges them with the local state, and saves the updated list to CSV.

8. Sequence diagrams

The following sequence diagrams show the main runtime interactions between the user interface, services, data model, and persistence/API layers.

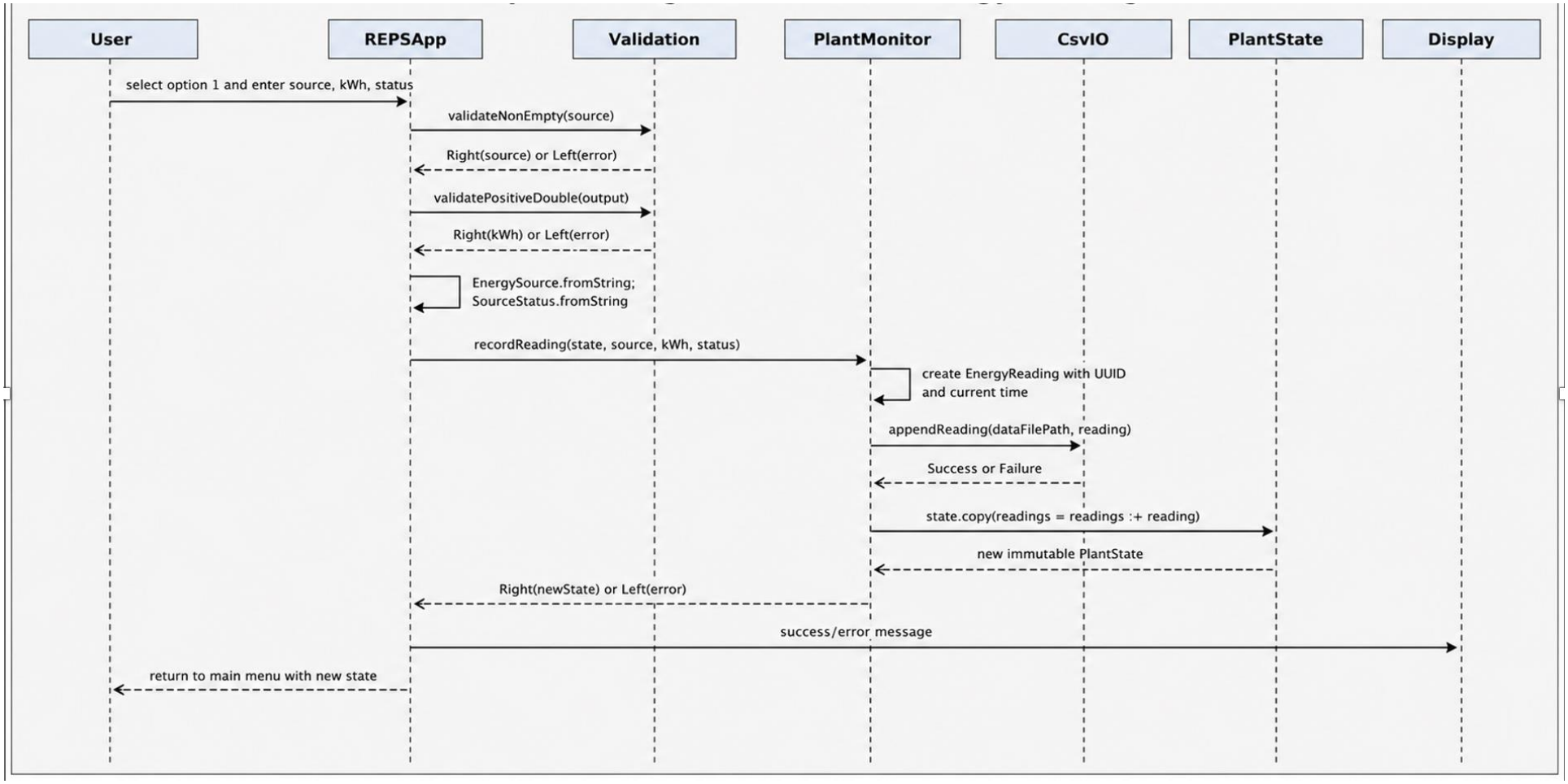

Figure 1. Recording a new energy reading.

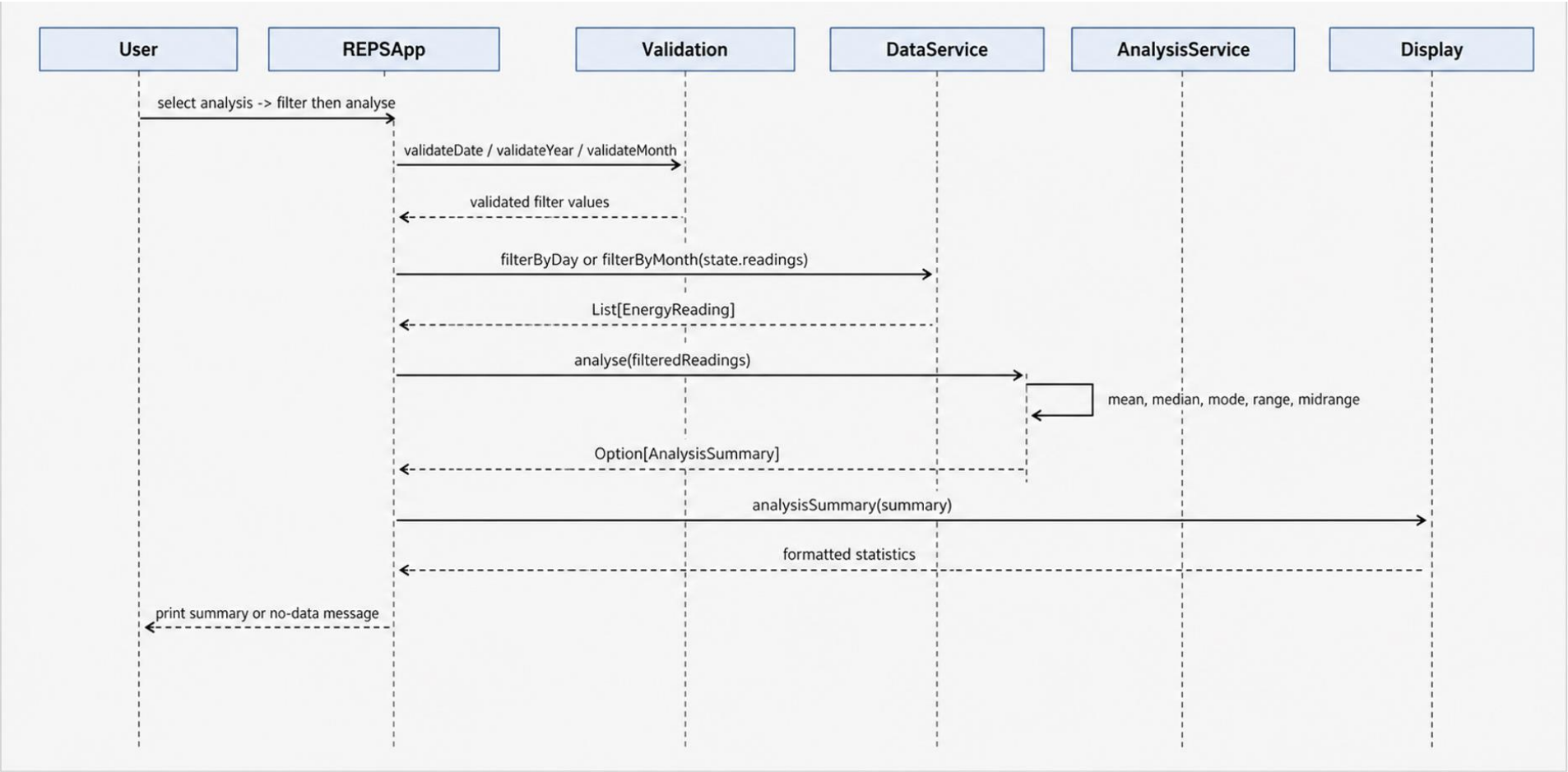

Figure 2. Filtering readings and generating an analysis summary.

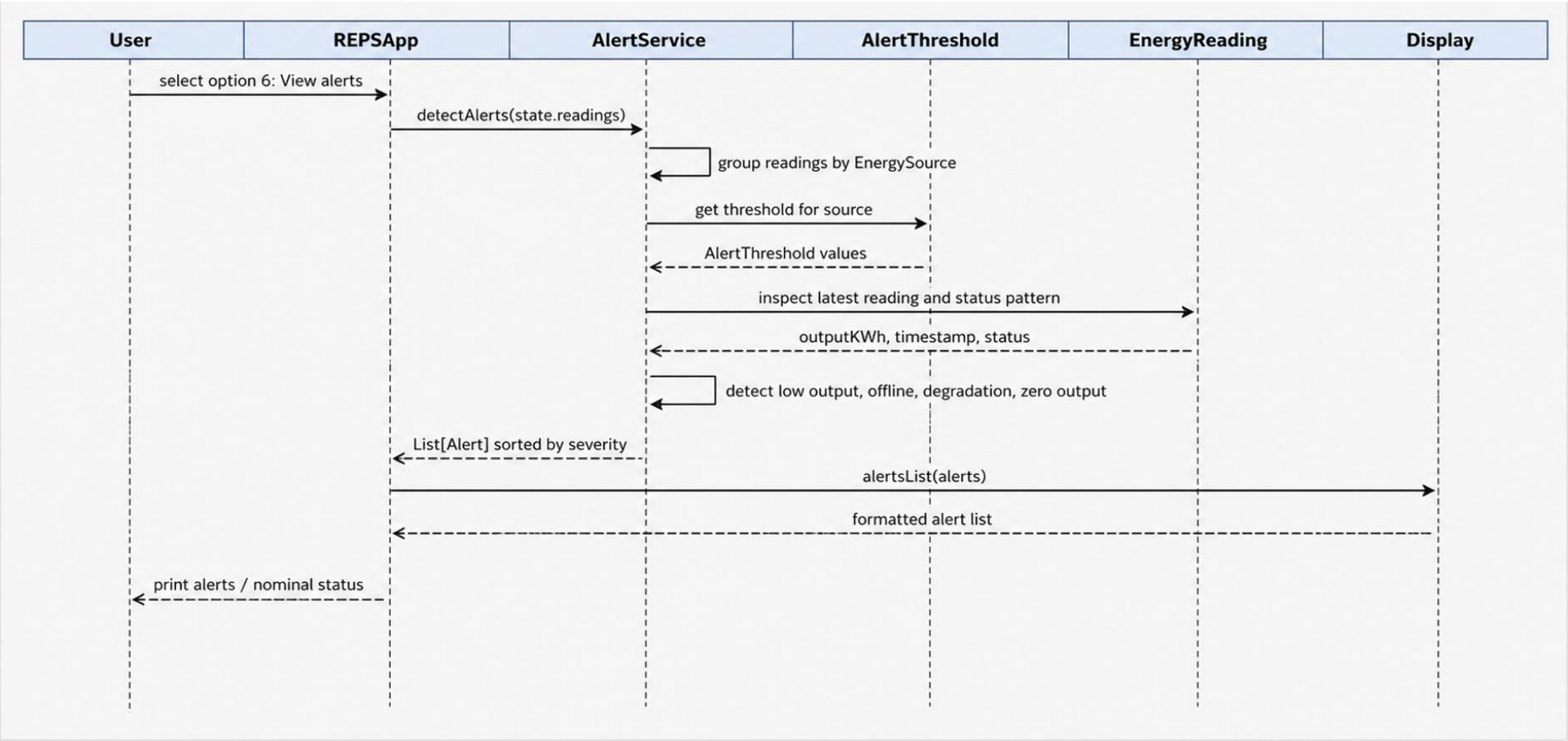

Figure 3. Generating alerts from current readings.

9. Class diagram

The class diagram summarises the main Scala objects, case classes, sealed traits, and their dependencies.

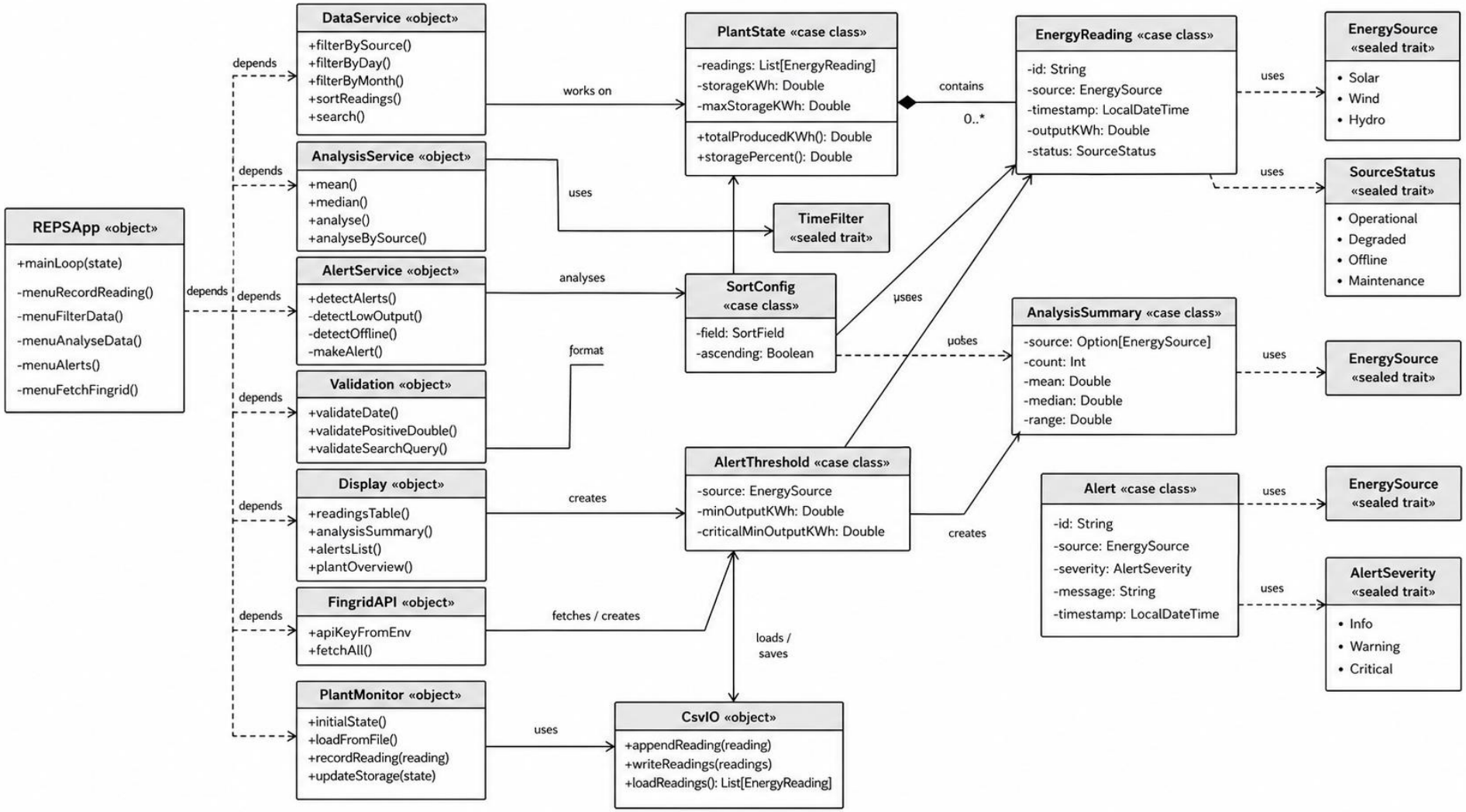

Figure 4. REPS class diagram.

10. Validation and error handling

User input validation is centralised in Validation. Methods such as validateDate, validatePositiveDouble, validateYear, validateMonth, and validateHour return Either[String, A], so invalid input can be handled without throwing exceptions. Menu handlers use for-comprehensions to combine multiple validation steps. If any step fails, the result becomes Left(error), and the application prints a clear error message while keeping the old PlantState.

File loading returns Either[String, List[EnergyReading]]. Appending/writing CSV uses Try because it wraps Java I/O exceptions. FingridAPI also returns Either so network and parsing failures can be reported to the user safely.

11. Testing notes and limitations

- The codebase includes scalatest as a dependency, so unit tests can be added for AnalysisService, DataService, Validation, CsvIO parsing, and AlertService.
- Fingrid live data requires a valid API key and network access. Without the key, the rest of the local CSV-based functionality can still be used.
- The outputKWh field is also used for Fingrid MW values for compatibility with the existing model.
- Alert ordering is currently sorted by severity text, which places CRITICAL before WARNING due to the string order used by toString values.

12. How to run the system

Prerequisites: Java 11+ and sbt 1.9+ installed.

```
git clone github.com/arsoykot/reps
```

```
cd reps/src/main/scala
```

```
sbt run
```

Optional Fingrid API use: set FINGRID_API_KEY as an environment variable or enter the key when prompted in the application.

AI Declaration: ChatGPT and Claude is used to refine the report and correct grammatical errors. Draw.io is used to edit the diagram and the text within it.